Problem 3

In [1]:

```
import numpy as np
```

Training (Constructing Weights)

In [8]:

```
states = np.array([[+1, -1, +1, -1, +1], [-1, +1, +1, -1, +1]])

# Number of states
N = len(states[0])
M = len(states)
# Construct the weight matrix
W = 1 / N * states.T @ states
for i in range(N):
    W[i][i] = 0
W
```

Out[8]:

```
array([[ 0. , -0.4,  0. ,  0. ,  0. ],
       [-0.4,  0. ,  0. ,  0. ,  0. ],
       [ 0. ,  0. ,  0. , -0.4,  0.4],
       [ 0. ,  0. , -0.4,  0. , -0.4],
       [ 0. ,  0. ,  0.4, -0.4,  0. ]])
```

Probing Pattern

In [3]:

```
initial_input_state = np.array([+1, +1, +1, +1, +1])
```

Dynamic Evolution

In [19]:

```
# Hard-limiting non-linearity function
f_h = np.vectorize(lambda x: 1 if x >= 0 else -1)

# Dynamic evolution
def evolve(order, log=False):
    t = 0
    last_U = None
    U = initial_input_state.copy()
    if log:
        print('U({}) = {}'.format(t, U))

    # Until convergence
    while not np.array_equal(U, last_U):
        t += 1
        last_U = U.copy()
        # Update the nodes in order
        for i in order:
            # Be sure to zero-index

            # YOUR CODE HERE
            U[i-1] = f_h(sum(W[j][i-1] * U[j] for j in range(N) if j != i-1))
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
            if log:
                print('U_{}({}) = {}'.format(i, t, U[i-1]))

        # Log U for this iteration
        if log:
            print('U({}) = {}'.format(t, U))
            None

    return U
```

In [21]:

```
U = evolve([3, 1, 5, 2, 4], log=True)
print(U)
```

```
U(0) = [1 1 1 1 1]
U_3(1) = 1
U_1(1) = -1
U_5(1) = 1
U_2(1) = 1
U_4(1) = -1
U(1) = [-1  1  1 -1  1]
U_3(2) = 1
U_1(2) = -1
U_5(2) = 1
U_2(2) = 1
U_4(2) = -1
U(2) = [-1  1  1 -1  1]
[-1  1  1 -1  1]
```

In [22]:

```
U = evolve([2, 4, 3, 5, 1], log=True)
print(U)
```

```
U(0) = [1 1 1 1 1]
U_2(1) = -1
U_4(1) = -1
U_3(1) = 1
U_5(1) = 1
U_1(1) = 1
U(1) = [ 1 -1  1 -1  1]
U_2(2) = -1
U_4(2) = -1
U_3(2) = 1
U_5(2) = 1
U_1(2) = 1
U(2) = [ 1 -1  1 -1  1]
[ 1 -1  1 -1  1]
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

🧠 (Problem 4) Traveling Salesman Problem — Hopfield Neural Network

Interactive Classroom Demo

Run the final cell and watch the network solve the TSP live — no widgets needed.

Panel	What it shows
🗺️ **Route Map**	The inferred tour evolving on a geographic map
🔥 **V Matrix**	Neuron activations sharpening toward a valid permutation
⚡ **Energy Curve**	Network energy dropping as it converges

In [20]:

```
import numpy as np
import matplotlib
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
from matplotlib.colors import LinearSegmentedColormap
from IPython.display import clear_output
import time
import warnings
warnings.filterwarnings('ignore')

print('Libraries loaded ✓')
```

```
Libraries loaded ✓
```

City Setup & Distance Matrix

In [21]:

```
cities = ['San Diego', 'Los Angeles', 'Chicago', 'New York', 'Boston', 'Seattle']
N = len(cities)

coords = np.array([
    [-117.2, 32.7],
    [-118.2, 34.1],
    [ -87.6, 41.9],
    [ -74.0, 40.7],
    [ -71.1, 42.4],
    [-122.3, 47.6],
])

D_raw = np.zeros((N, N))
for i in range(N):
    for j in range(N):
        D_raw[i, j] = np.sqrt(np.sum((coords[i] - coords[j])**2))
D_norm = D_raw / D_raw.max()

CITY_COLORS = [
    '#FF6B6B', # San Diego
    '#4ECDC4', # Los Angeles
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
    '#45B7D1', # Chicago
    '#96CEB4', # New York
    '#FFEAA7', # Boston
    '#FF6B6B', # Seattle
]

BG     = '#1a1a2e'
PANEL  = '#16213e'
LBLUE  = '#4ECDC4'
GREEN  = '#0ead69'
YELLOW = '#f5a623'
ACCENT = '#e94560'

VMAP = LinearSegmentedColormap.from_list(
    'vmap', ['#1a1a2e', '#e94560', '#f5a623', '#ffffff'], N=256
)

print('Cities configured ✓')
```

```
Cities configured ✓
```

Hopfield Network

In [22]:

```
class HopfieldTSP:
    def __init__(self, N, D, A=500, B=500, C=200, D_coeff=100, dt=1e-4, gain=50):
        self.N, self.D = N, D
        self.A, self.B, self.C, self.D_coeff = A, B, C, D_coeff
        self.dt, self.gain = dt, gain
        self.U = 0.02 * np.random.randn(N, N)
        self.V = self._sig(self.U)

    def _sig(self, u):
        return 0.5 * (1 + np.tanh(u * self.gain / 2))

    def step(self):
        V, N = self.V, self.N
        dU = np.zeros((N, N))
        total = np.sum(V)
        for i in range(N):
            for j in range(N):
                jn, jp = (j+1)%N, (j-1)%N
                dU[i,j] = (
                    - self.A * (np.sum(V[i,:]) - V[i,j])
                    - self.B * (np.sum(V[:,j]) - V[i,j])
                    - self.C * (total - N)
                    - self.D_coeff * np.dot(self.D[i,:], V[:,jn] + V[:,jp])
                )
        self.U += self.dt * (-self.U + dU)
        self.V = self._sig(self.U)

    def energy(self):
        V, N = self.V, self.N
        E  = self.A/2 * sum((np.sum(V[i,:])**2 - np.sum(V[i,:]**2)) for i in range(N))
        E += self.B/2 * sum((np.sum(V[:,j])**2 - np.sum(V[:,j]**2)) for j in range(N))
        E += self.C/2 * (np.sum(V) - N)**2
        d  = sum(self.D[i,:] @ V[:,(j+1)%N] * V[i,j] for i in range(N) for j in range(N))
        E += self.D_coeff/2 * d
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
        return E

    def decode_tour(self):
        t = np.argmax(self.V, axis=0).tolist()
        return t if len(set(t)) == self.N else None

print('HopfieldTSP defined ✓')
```

```
HopfieldTSP defined ✓
```

▶ Run This Cell to Watch the Network Converge Live

In [29]:

```
# ── Settings ──────────────────────────────────────────────────────
N_STEPS      = 3000   # total network iterations
DISPLAY_EVERY = 150   # show a frame every N steps  ← adjust for speed
FRAME_DELAY  = 0.1    # seconds between frames      ← adjust for speed

# ── Find a seed that produces a valid tour ────────────────────────
# print('Searching for a good random seed...')
# good_seed = 0
# for seed in np.random.randint(0, 1000000, 100):
#     np.random.seed(seed)
#     test = HopfieldTSP(N, D_norm)
#     for _ in range(N_STEPS):
#         test.step()
#     if test.decode_tour() is not None:
#         good_seed = seed
#         print(f'Seed {seed} gives a valid tour ✓')
#         break

# ── Main run with live plotting ───────────────────────────────────
good_seed = 80163
np.random.seed(good_seed)
net = HopfieldTSP(N, D_norm)

energy_history = []
step_history   = []
frame_num      = 0

display_steps = list(range(1, DISPLAY_EVERY, 10))          # a few early frames
display_steps += list(range(DISPLAY_EVERY, N_STEPS+1, DISPLAY_EVERY))
display_steps = sorted(set(display_steps))

for s in range(1, N_STEPS + 1):
    net.step()
    e = net.energy()
    energy_history.append(e)
    step_history.append(s)

    if s not in display_steps:
        continue

    frame_num += 1
    tour  = net.decode_tour()
    V     = net.V
    valid = tour is not None
    status_color = GREEN  if valid else YELLOW
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
    status_text  = '✓  VALID TOUR FOUND' if valid else '⟳  Converging…'
    sharpness = (np.mean(np.max(V, axis=1)) + np.mean(np.max(V, axis=0))) / 2

    # ── Build figure ──────────────────────────────────────────────
    fig = plt.figure(figsize=(17, 9), facecolor=BG)
    gs  = gridspec.GridSpec(
        2, 3, figure=fig,
        left=0.05, right=0.97, top=0.88, bottom=0.09,
        wspace=0.38, hspace=0.52
    )

    # Title
    fig.text(0.5, 0.95,
             '🧠  Hopfield Neural Network — TSP Convergence',
             ha='center', fontsize=16, fontweight='bold', color='white')
    fig.text(0.5, 0.906,
             f'Step {s:>4d} / {N_STEPS}          {status_text}',
             ha='center', fontsize=12, color=status_color, fontweight='bold')

    # ── Panel A: Route Map ────────────────────────────────────────
    ax_map = fig.add_subplot(gs[0, :2])
    ax_map.set_facecolor(PANEL)
    for sp in ax_map.spines.values(): sp.set_edgecolor('#333366')

    x, y = coords[:,0], coords[:,1]

    if valid:
        for i in range(N):
            c1, c2 = tour[i], tour[(i+1)%N]
            ax_map.annotate('',
                xy=(x[c2], y[c2]), xytext=(x[c1], y[c1]),
                arrowprops=dict(arrowstyle='->', color=LBLUE, lw=2.5,
                                connectionstyle='arc3,rad=0.12', mutation_scale=20))
    else:
        for i in range(N):
            for j in range(N):
                for k in range(N):
                    w = V[i,j] * V[k,(j+1)%N]
                    if w > 0.06:
                        ax_map.plot([x[i], x[k]], [y[i], y[k]],
                                    color=LBLUE, alpha=min(w*1.5, 0.5), lw=1.5)

    for i in range(N):
        ax_map.scatter(x[i], y[i], s=300, color=CITY_COLORS[i],
                       zorder=6, edgecolors='white', linewidths=1.8)
        ax_map.text(x[i], y[i]+0.85, cities[i],
                    ha='center', fontsize=10, fontweight='bold', color='white', zorder=7)
        if valid and i in tour:
            ax_map.text(x[i], y[i]-1.1, f'#{tour.index(i)+1}',
                        ha='center', fontsize=9, color=CITY_COLORS[i],
                        fontweight='bold', zorder=7)

    ax_map.set_xlim(-124, -62)
    ax_map.set_ylim(30, 50)
    ax_map.set_title('🗺  Route Map', fontsize=12, fontweight='bold', color='white', pad
    ax_map.set_xlabel('Longitude', fontsize=9, color='#aaaacc')
    ax_map.set_ylabel('Latitude',  fontsize=9, color='#aaaacc')
    ax_map.grid(True, alpha=0.12, color='white')
    ax_map.tick_params(colors='#aaaacc', labelsize=8)
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
    if valid:
        seq = '  →  '.join(cities[c] for c in tour) + f'  →  {cities[tour[0]]}'
        ax_map.text(0.5, -0.15, seq, transform=ax_map.transAxes,
                    ha='center', fontsize=9, color=LBLUE, fontweight='bold')
    else:
        ax_map.text(0.5, -0.15, 'Route undetermined — neurons still competing',
                    transform=ax_map.transAxes,
                    ha='center', fontsize=9, color=YELLOW, style='italic')

    # ── Panel B: V Matrix ──────────────────────────────────────────
    ax_v = fig.add_subplot(gs[0, 2])
    ax_v.set_facecolor(PANEL)

    im = ax_v.imshow(V, cmap=VMAP, vmin=0, vmax=1, aspect='auto')
    for i in range(N):
        for j in range(N):
            val = V[i, j]
            ax_v.text(j, i, f'{val:.2f}',
                      ha='center', va='center', fontsize=9,
                      color='white' if val < 0.6 else '#1a1a2e',
                      fontweight='bold' if val > 0.7 else 'normal')

    ax_v.set_xticks(range(N))
    ax_v.set_xticklabels([f'Pos {k+1}' for k in range(N)], fontsize=8, color='#aaaacc')
    ax_v.set_yticks(range(N))
    ax_v.set_yticklabels([c[:9] for c in cities], fontsize=8, color='#aaaacc')
    ax_v.set_title('🔥  V Matrix  [city × position]',
                   fontsize=10, fontweight='bold', color='white', pad=7)
    cb = plt.colorbar(im, ax=ax_v, fraction=0.046, pad=0.04)
    cb.ax.tick_params(colors='#aaaacc', labelsize=7)
    cb.set_label('Activation', color='#aaaacc', fontsize=7)

    ax_v.text(0.5, -0.13,
              f'Sharpness: {sharpness:.3f}  (1.0 = perfect assignment)',
              transform=ax_v.transAxes, ha='center', fontsize=8,
              color=GREEN if sharpness > 0.9 else YELLOW)

    # ── Panel C: Energy Curve ──────────────────────────────────────
    ax_e = fig.add_subplot(gs[1, :])
    ax_e.set_facecolor(PANEL)
    for sp in ax_e.spines.values(): sp.set_edgecolor('#333366')

    ax_e.plot(step_history, energy_history, color=LBLUE, lw=2)
    ax_e.fill_between(step_history, energy_history, alpha=0.08, color=LBLUE)
    ax_e.scatter([s], [e], s=140, color=status_color,
                 zorder=6, edgecolors='white', linewidths=1.5)
    ax_e.annotate(f'  E = {e:.1f}', (s, e),
                  fontsize=9, color=status_color, fontweight='bold', va='center')

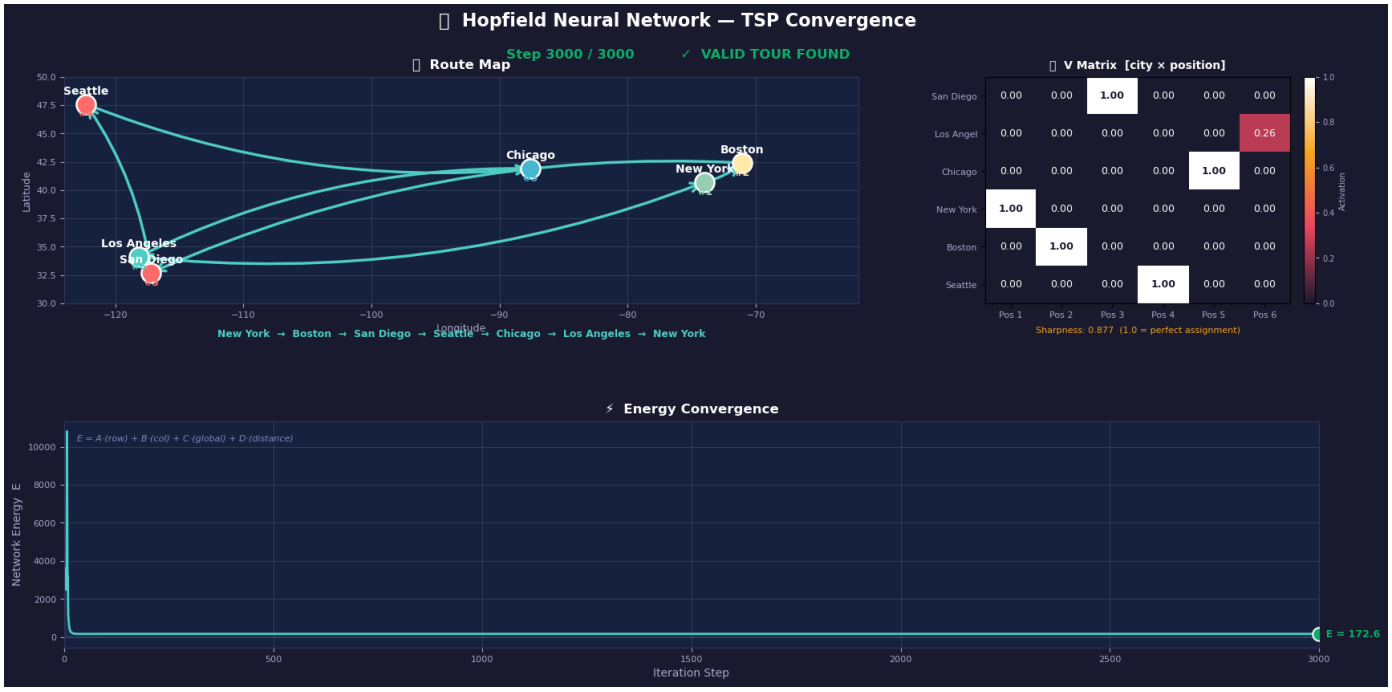
    ax_e.set_xlim(0, N_STEPS)
    ax_e.set_xlabel('Iteration Step', fontsize=10, color='#aaaacc')
    ax_e.set_ylabel('Network Energy  E', fontsize=10, color='#aaaacc')
    ax_e.set_title('⚡  Energy Convergence',
                   fontsize=12, fontweight='bold', color='white', pad=7)
    ax_e.grid(True, alpha=0.12, color='white')
    ax_e.tick_params(colors='#aaaacc', labelsize=8)
    ax_e.text(0.01, 0.91,
              'E = A·(row) + B·(col) + C·(global) + D·(distance)',
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
            transform=ax_e.transAxes, fontsize=8,
            color='#8888bb', style='italic')

    clear_output(wait=True)
    plt.show()
    time.sleep(FRAME_DELAY)

print('\n✅ Done! Scroll up to review all convergence frames.')
```




```
✅ Done! Scroll up to review all convergence frames.
```

In [30]:

```
net.decode_tour()
```

Out[30]:

```
[3, 4, 0, 5, 2, 1]
```

📖 Teaching Notes

Stage	V Matrix	Route Map	Energy
Early	All cells ≈ equal — neurons uncertain	Soft overlapping connections	High, noisy
Mid	Some cells brightening — competition starting	Structure emerging, may have conflicts	Dropping steeply
Late	One bright cell per row & column — valid permutation	Single clean tour with stop numbers	Plateau reached

Adjust `DISPLAY_EVERY` and `FRAME_DELAY` in the run cell to control the speed of the demo.

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Problem 5

In [1]:

```
import os
os.environ['KMP_DUPLICATE_LIB_OK'] = 'TRUE'
```

In [2]:

```
import string
import random
import torch
import torch.nn as nn
import matplotlib.pyplot as plt
import numpy as np
```

Prepare for Dataset

In [3]:

```
# Get a random sequence of sine curve.
def get_random_seq():
    seq_len     = 128  # The length of an input sequence.
    # Sample a sequence.
    t = np.arange(0, seq_len)
    a = 2*np.pi*1.0/seq_len
    b = 2*np.pi*np.random.rand()*5
    seq = np.sin(a*t+b)
    return seq

# Sample a mini-batch including input tensor and target tensor.
def get_input_and_target():
    seq    = get_random_seq()
    input  = torch.tensor(seq[:-1]).float().view(-1,1,1) # Input sequence.
    target = torch.tensor(seq[1:]).float().view(-1,1,1)  # Target sequence.
    return input, target
```

Choose a Device

In [4]:

```
# If there are GPUs, choose the first one for computing. Otherwise use CPU.
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
print(device)
# If 'cuda:0' is printed, it means GPU is available.
```

```
cpu
```

Network Definition

In [5]:

```
class Net(nn.Module):
    def __init__(self):
        # Initialization.
        super(Net, self).__init__()
        self.input_size  = 1
        self.hidden_size = 100
        self.output_size = 1
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
        self.rnn_cell = nn.RNNCell(self.input_size, self.hidden_size)
        self.linear   = nn.Linear(self.hidden_size, self.output_size)

    def forward(self, input, hidden):
        """ Forward function.
              input:  Input. It refers to the x_t in homework write-up.
              hidden: Previous hidden state. It refers to the h_{t-1}.
            Returns (output, hidden) where output refers to y_t and
                     hidden refers to h_t.
        """
        # Forward function.
        h_t = self.rnn_cell(input, hidden)
        y_t = self.linear(h_t)
        return y_t, h_t

    def init_hidden(self):
        # Initial hidden state.
        # 1 means batch size = 1.
        return torch.zeros(1, self.hidden_size).to(device)

net = Net()     # Create the network instance.
net.to(device)  # Move the network parameters to the specified device.
```

Out[5]:

```
Net(
  (rnn_cell): RNNCell(1, 100)
  (linear): Linear(in_features=100, out_features=1, bias=True)
)
```

Training Step and Evaluation Step

In [6]:

```
loss_func = lambda y, t: (y - t)**2

# Training step function.
def train_step(net, opt, input, target):
    """ Training step.
        net:    The network instance.
        opt:    The optimizer instance.
        input:  Input tensor.  Shape: [seq_len, 1, 1].
        target: Target tensor. Shape: [seq_len, 1].
    """
    seq_len = input.shape[0]    # Get the sequence length of current input.
    hidden = net.init_hidden()  # Initial hidden state.
    net.zero_grad()             # Clear the gradient.
    loss = 0                    # Initial loss.

    for t in range(seq_len):    # For each one in the input sequence.
        output, hidden = net(input[t], hidden)
        loss += loss_func(output, target[t])

    loss.backward()             # Backward.
    opt.step()                  # Update the weights.

    return loss / seq_len       # Return the average loss w.r.t sequence length.
```

In [7]:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
# Evaluation step function.
def eval_step(net, predicted_len=100):
    # Initialize the hidden state, input and the predicted sequence.
    hidden        = net.init_hidden()
    init_seq      = get_random_seq()
    init_input    = torch.tensor(init_seq).float().view(-1,1,1).to(device)
    predicted_seq = []

    # Use initial points on the curve to "build up" hidden state.
    for t in range(len(init_seq) - 1):
        output, hidden = net(init_input[t], hidden)

    # Set current input as the last character of the initial string.
    input = init_input[-1]

    # Predict more points after the initial string.
    for t in range(predicted_len):
        # Get the current output and hidden state.
        output, hidden = net(input, hidden)

        # Add predicted point to the sequence and use it as next input.
        predicted_seq.append(output.item())

        # Use the predicted point to generate the input of next round.
        input = output

    return init_seq, predicted_seq
```

Training Procedure

In [8]:

```
# Number of iterations.
iters       = 200    # Number of training iterations.
print_iters = 10     # Number of iterations for each log printing.

# The loss variables.
all_losses = []
loss_sum   = 0

# Initialize the optimizer and the loss function.
opt       = torch.optim.Adam(net.parameters(), lr=0.005)
loss_func = nn.MSELoss()

# Training procedure.
for i in range(iters):
    input, target = get_input_and_target()            # Fetch input and target.
    input, target = input.to(device), target.to(device) # Move to GPU memory.
    loss      = train_step(net, opt, input, target)   # Calculate the loss.
    loss_sum += loss                                  # Accumulate the loss.

    # Print the log.
    if i % print_iters == print_iters - 1:
        print('iter:{}/{} loss:{}'.format(i, iters, loss_sum / print_iters))
        #print('generated sequence: {}\n'.format(eval_step(net)))

        # Track the loss.
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

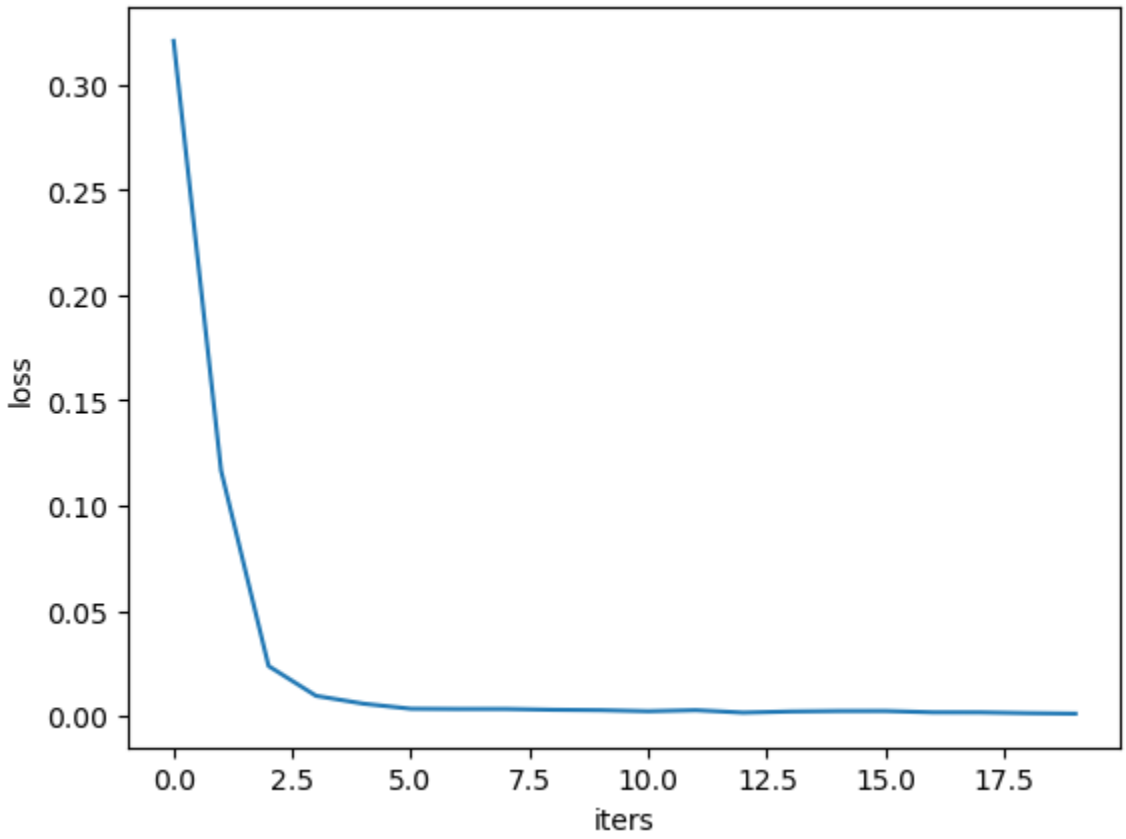
```
        all_losses.append((loss_sum / print_iters).item())
        loss_sum = 0
```

```
iter:9/200 loss:0.32080748677253723
iter:19/200 loss:0.11659383773803711
iter:29/200 loss:0.023761939257383347
iter:39/200 loss:0.009609444066882133
iter:49/200 loss:0.005872826091945171
iter:59/200 loss:0.003459220752120018
iter:69/200 loss:0.0033439001999795437
iter:79/200 loss:0.003350761253386736
iter:89/200 loss:0.0029689432121813297
iter:99/200 loss:0.002794354921206832
iter:109/200 loss:0.002278412925079465
iter:119/200 loss:0.002791750244796276
iter:129/200 loss:0.0016406966606155038
iter:139/200 loss:0.0021623976062983274
iter:149/200 loss:0.002376296790316701
iter:159/200 loss:0.002387423301115632
iter:169/200 loss:0.0017720026662573218
iter:179/200 loss:0.001747651258483529
iter:189/200 loss:0.0013632839545607567
iter:199/200 loss:0.001131489989347756
```

Training Loss Curve

In [9]:

```
plt.xlabel('iters')
plt.ylabel('loss')
plt.plot(all_losses)
plt.show()
```

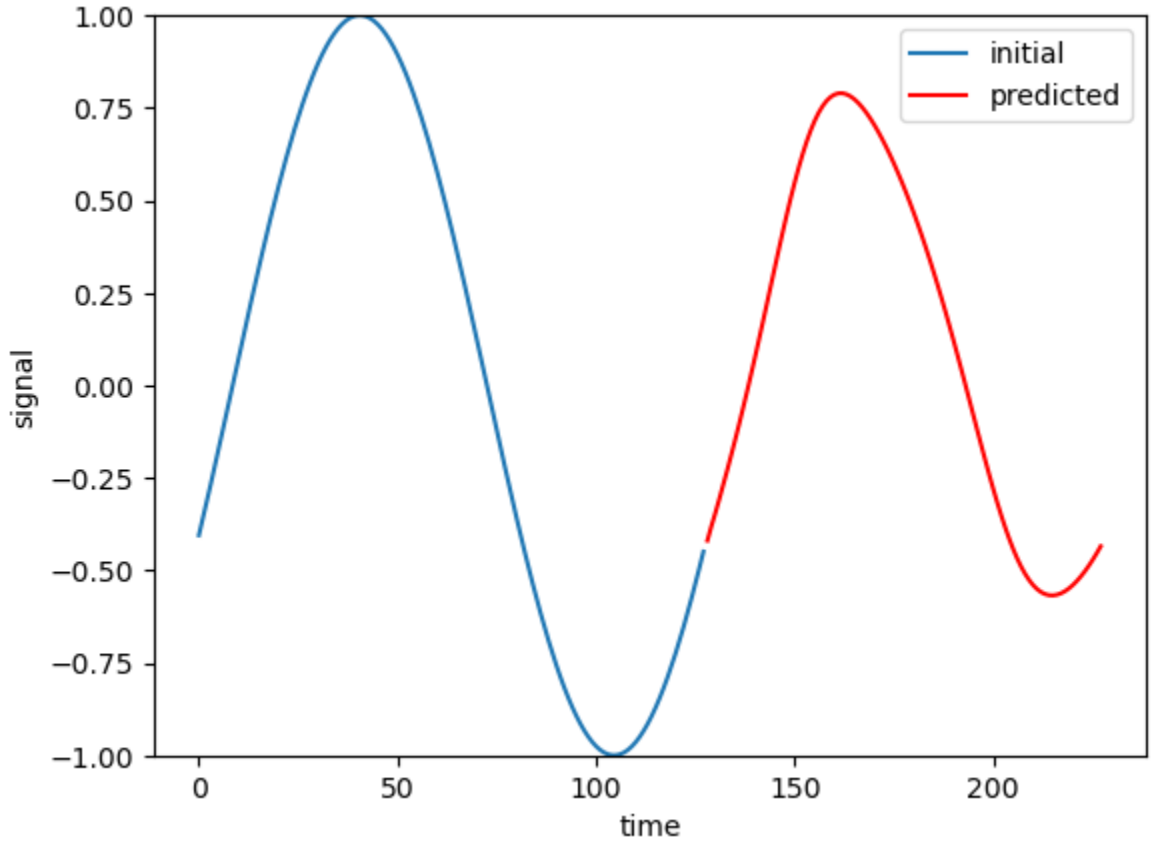


Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Evaluation: A Sample of Generated Sequence

In [10]:

```
init_seq, predicted_seq = eval_step(net, predicted_len=100)
init_t      = np.arange(0, len(init_seq))
predicted_t = np.arange(len(init_seq), len(init_seq)+len(predicted_seq))
plt.plot(init_t, init_seq, label='initial')
plt.plot(predicted_t, predicted_seq, color='red', label='predicted')
plt.legend()
plt.ylim([-1, 1])
plt.xlabel('time')
plt.ylabel('signal')
plt.show()
```



Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

Problem 6

In [1]:
```
import os
os.environ['KMP_DUPLICATE_LIB_OK'] = 'TRUE'
```

In [2]:
```
import string
import random
import torch
import torch.nn as nn
import matplotlib.pyplot as plt
```

Prepare for Dataset

In [3]:
```
all_chars       = string.printable
n_chars         = len(all_chars)
file            = open('./shakespeare.txt').read()
file_len        = len(file)

print('Length of file: {}'.format(file_len))
print('All possible characters: {}'.format(all_chars))
print('Number of all possible characters: {}'.format(n_chars))
```

```
Length of file: 1115394
All possible characters: 0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVW
XYZ!"#$%&'()*+,-./:;<=>?@[\]^_`{|}~

Number of all possible characters: 100
```

In [4]:
```
# Get a random sequence of the Shakespeare dataset.
def get_random_seq():
    seq_len     = 128  # The length of an input sequence.
    start_index = random.randint(0, file_len - seq_len)
    end_index   = start_index + seq_len + 1
    return file[start_index:end_index]

# Convert the sequence to one-hot tensor.
def seq_to_onehot(seq):
    tensor = torch.zeros(len(seq), 1, n_chars)
    # Shape of the tensor:
    #     (sequence length, batch size, classes)
    # Here we use batch size = 1 and classes = number of unique characters.
    for t, char in enumerate(seq):
        index = all_chars.index(char)
        tensor[t][0][index] = 1
    return tensor

# Convert the sequence to index tensor.
def seq_to_index(seq):
    tensor = torch.zeros(len(seq), 1)
    # Shape of the tensor:
    #     (sequence length, batch size).
    # Here we use batch size = 1.
```

```
    for t, char in enumerate(seq):
        tensor[t] = all_chars.index(char)
    return tensor

# Sample a mini-batch including input tensor and target tensor.
def get_input_and_target():
    seq    = get_random_seq()
    input  = seq_to_onehot(seq[:-1])      # Input is represented in one-hot.
    target = seq_to_index(seq[1:]).long() # Target is represented in index.
    return input, target
```

Choose a Device

In [5]:
```
# If there are GPUs, choose the first one for computing. Otherwise use CPU.
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
print(device)
# If 'cuda:0' is printed, it means GPU is available.
```

```
cpu
```

Network Definition

In [6]:
```
class Net(nn.Module):
    def __init__(self):
        # Initialization.
        super(Net, self).__init__()
        self.input_size  = n_chars   # Input size: Number of unique chars.
        self.hidden_size = 100       # Hidden size: 100.
        self.output_size = n_chars   # Output size: Number of unique chars.

        self.rnn_cell = nn.RNNCell(self.input_size, self.hidden_size)
        self.linear   = nn.Linear(self.hidden_size, self.output_size)

    def forward(self, input, hidden):
        """ Forward function.
              input:  One-hot input. It refers to the x_t in homework write-up.
              hidden: Previous hidden state. It refers to the h_{t-1}.
            Returns (output, hidden) where output refers to y_t and
                     hidden refers to h_t.
        """
        # Forward function.
        hidden = self.rnn_cell(input, hidden)
        output = self.linear(hidden)

        return output, hidden

    def init_hidden(self):
        # Initial hidden state.
        # 1 means batch size = 1.
        return torch.zeros(1, self.hidden_size).to(device)

net = Net()     # Create the network instance.
net.to(device)  # Move the network parameters to the specified device.
```

Out[6]:
```
Net(
  (rnn_cell): RNNCell(100, 100)
  (linear): Linear(in_features=100, out_features=100, bias=True)
)
```

Training Step and Evaluation Step

In [7]:
```python
loss_func = nn.CrossEntropyLoss()

# Training step function.
def train_step(net, opt, input, target):
    """ Training step.
        net:    The network instance.
        opt:    The optimizer instance.
        input:  Input tensor.  Shape: [seq_len, 1, n_chars].
        target: Target tensor. Shape: [seq_len, 1].
    """
    seq_len = input.shape[0]    # Get the sequence length of current input.
    hidden = net.init_hidden()  # Initial hidden state.
    net.zero_grad()             # Clear the gradient.
    loss = 0                    # Initial loss.

    for t in range(seq_len):    # For each one in the input sequence.
        output, hidden = net(input[t], hidden)
        loss += loss_func(output, target[t])

    loss.backward()             # Backward.
    opt.step()                  # Update the weights.

    return loss / seq_len       # Return the average loss w.r.t sequence length.
```

In [8]:
```python
# Evaluation step function.
def eval_step(net, init_seq='W', predicted_len=100):
    # Initialize the hidden state, input and the predicted sequence.
    hidden        = net.init_hidden()
    init_input    = seq_to_onehot(init_seq).to(device)
    predicted_seq = init_seq

    # Use initial string to "build up" hidden state.
    for t in range(len(init_seq) - 1):
        output, hidden = net(init_input[t], hidden)

    # Set current input as the last character of the initial string.
    input = init_input[-1]

    # Predict more characters after the initial string.
    for t in range(predicted_len):
        # Get the current output and hidden state.
        output, hidden = net(input, hidden)

        # Sample from the output as a multinomial distribution.
        predicted_index = torch.multinomial(output.view(-1).exp(), 1)[0]

        # Add predicted character to the sequence and use it as next input.
        predicted_char  = all_chars[predicted_index]
```

```
        predicted_seq  += predicted_char

        # Use the predicted character to generate the input of next round.
        input = seq_to_onehot(predicted_char)[0].to(device)

    return predicted_seq
```

Training Procedure

In [9]:
```
# Number of iterations.
iters       = 20000  # Number of training iterations.
print_iters = 100    # Number of iterations for each log printing.

# The loss variables.
all_losses = []
loss_sum   = 0

# Initialize the optimizer and the loss function.
opt       = torch.optim.Adam(net.parameters(), lr=0.005)
loss_func = nn.CrossEntropyLoss()

# Training procedure.
for i in range(iters):
    input, target = get_input_and_target()            # Fetch input and target.
    input, target = input.to(device), target.to(device) # Move to GPU memory.
    loss      = train_step(net, opt, input, target)   # Calculate the loss.
    loss_sum += loss                                  # Accumulate the loss.

    # Print the log.
    if i % print_iters == print_iters - 1:
        print('iter:{}/{} loss:{}'.format(i, iters, loss_sum / print_iters))
        print('generated sequence: {}\n'.format(eval_step(net)))

        # Track the loss.
        all_losses.append((loss_sum / print_iters).item())
        loss_sum = 0
```

```
iter:19799/20000 loss:1.7809001207351685
generated sequence: WO
TENRELANE:
You shears,
Preasoled
To recronthing as his spake,
To see, the rides will tencest ire h

iter:19899/20000 loss:1.7936503887176514
generated sequence: Wit
To drat us.

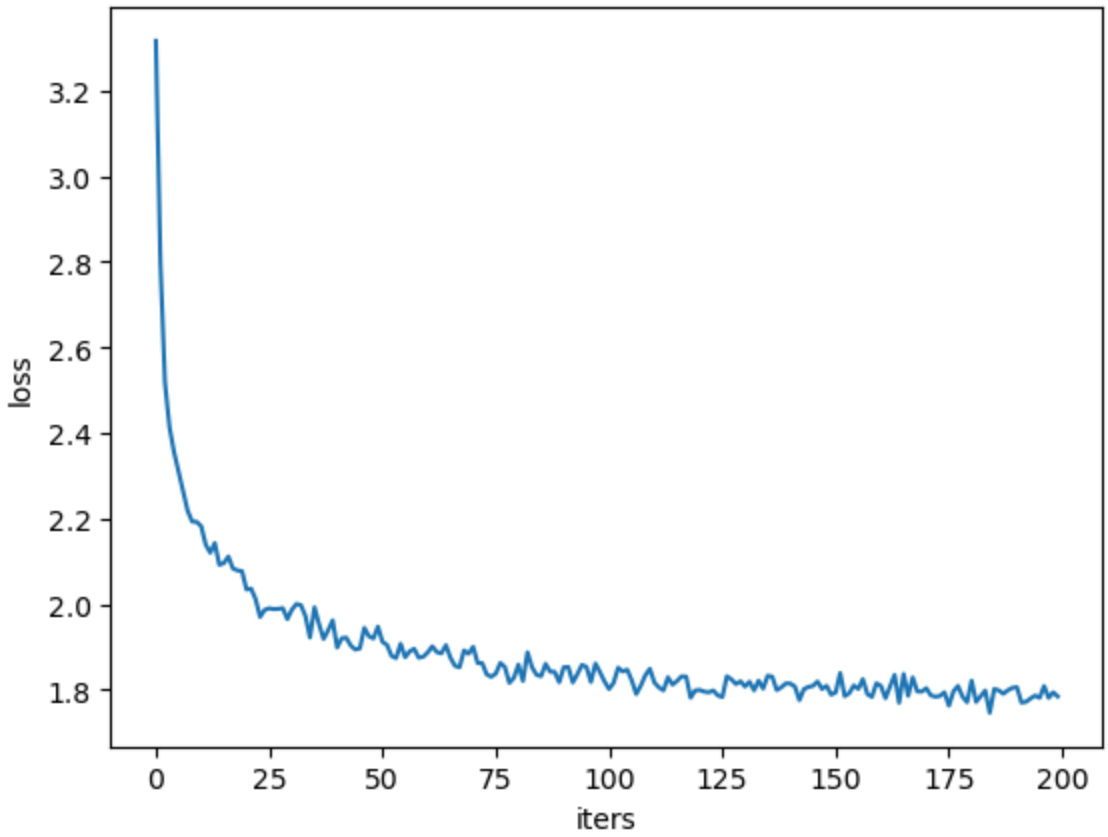
GLOUTES:
M' Kcem is 'Claider
Alm:s.
O, thou sir so LAUDE LORK:
They nothent's awhy,

iter:19999/20000 loss:1.7841639518737793
generated sequence: Why:
Most
rought out upon.

Lick aunt her wilt.
Ahe denverst, it may breads as the madien was
I jover
```

Training Loss Curve

In [10]:
```
plt.xlabel('iters')
plt.ylabel('loss')
plt.plot(all_losses)
plt.show()
```



Evaluation: A Sample of Generated Sequence

In [11]:
```
print(eval_step(net, predicted_len=600))
```

```
Whee,
Dook thinar: hath and groken
First,
In made but borss, the facery is;
Is this some EDthactry you malimanged with seevous; my
Fames; theh, bue be ote! good Andinurst to tringeinioblion:
my very poolly by thou nott.

LAUTER:
I you queer, my marraces for you, befice
Decouds but crow foo!
That stong by with it he'll oun nook, Buarh in liven'd of you my mest nome.

WARTIUMERD II:
Mean
Than hy've, and Enates an my pristess thriss to he's disfal'd.
For thy foow keet that you
Porry.

BUCKINGHAM: love my brolity ot hop youbpurding my liveng, solands:
Yen:
Fhall you, do you; his king
That he,
And b
```